

Research Companion

Full Documentation - Version 0.7.0

Overview, architecture, every feature, tech stack, and roadmap - the single consolidated reference.

This is the single consolidated reference for Research Companion: what it is, how it is built, every feature it ships, and where it is going. For task-oriented usage see `USER_MANUAL.md`; for the ingestion internals see `DEVELOPER_GUIDE.md`; for the plan of record see `ROADMAP.md`.

1. Overview & identity

Research Companion is a **local-first, open-source** tool for writing and evaluating research papers. You give it papers — your own draft and the literature around it — and a team of specialized agents reads them, builds a shared knowledge graph, and answers the questions every author and reviewer faces: *Are my references real? Is my contribution novel? Which papers support or threaten my draft, and where? Have I read everything my bibliography cites? Does the paper fit its venue? Is the work reproducible?*

Its single organizing principle is **verifiability**. Every verdict carries evidence you can check; every quote is verified verbatim against the source text; deterministic computation is preferred over LLM guesswork; and when something cannot be verified, the tool says so on screen rather than hiding it.

Key properties

- **Local-first** — all state is plain files under `~/ .research-companion/`. No database, no required server beyond the local Lab.
- **Deterministic-first** — most checks (references, reproducibility, ethics, severity, strength, suggestions, timeline, retrieval fallback) run with no LLM and no network, so they are cheap, reproducible, and inspectable.
- **Verifiable** — grounded Q&A with `[S#]` citations, verbatim quote checking, char-span provenance to the exact source passage, an append-only event audit log, and reference validation against authoritative external records.
- **Honest about limits** — no misconduct/plagiarism claims, no semantic entailment, no autonomous code execution.

2. Installation & quickstart

```
pip install research-companion # base CLI
pip install "research-companion[server]" # + the web Lab (FastAPI/uvicorn)
pip install "research-companion[docling]" # + layout-aware parsing / OCR
```

```
# CLI quickstart
research-companion add https://arxiv.org/abs/2410.05779
research-companion build # LLM extraction (cached) + build the graph
research-companion view # interactive HTML graph
research-companion chat "what are the main approaches?"

# The Research Lab (browser workspace)
research-companion lab serve # http://127.0.0.1:8765
```

- **Python:** `>=3.10, <3.14`.
- **Keys:** `ANTHROPIC_API_KEY` (default provider) or `OPENAI_API_KEY`; optional `HF_TOKEN` for semantic search. Stored in a local `.env`, never committed.
- **Offline demos (zero-key):** `python examples/demo_offline.py` (full review + rebuttal), `python examples/demo_lab_offline.py` (Lab event replay).

3. End-to-end workflow

Research Companion spans the paper lifecycle:

1. **Ideation / organization** — isolated *Researches* (workspaces).
2. **Discovery** — discover (Semantic Scholar topic search + citation expansion).
3. **Ingestion** — add arXiv/DOI/S2 IDs or PDFs; folder ingest; Docling/pypdfium.
4. **Knowledge graph** — concepts/methods/datasets/claims/results merged across papers.
5. **Q&A** — quote-grounded answers with [S#] citations and reader jump-to-span.
6. **Citation verification** — coverage (bibliography as ground truth) + placement.
7. **Novelty & alignment** — claim-level novelty vs prior art; draft alignment.
8. **Reviewer-grade checks** — venue-fit, severity ranking, reproducibility, ethics.
9. **Rebuttal** — grounded point-by-point reviewer responses.

4. Architecture

4.1 Data flow

A paper flows: `**ingestion` → text extraction → sectioning → LLM structured extraction → graph delta → alignment/strength → retrieval → agents → report/UI.**

- **Ingestion** — `fetch.py::add_paper` pulls from arXiv/DOI/Semantic Scholar/local PDF. PDFs are parsed by `parsers/` (pypdfium2 default; docling optional).
- **Sectioning + extraction** — `sections.py` builds the section tree; `extract.py` does LLM structured entity extraction, cached on a prompt SHA.
- **Graph** — `graph.py::build_graph` merges per-paper entities into a cross-paper NetworkX graph persisted as `graph.json`.
- **Retrieval** — `qa.py` builds a section-unit index; `retrieve.py::rank_units`

ranks (BM25 + optional embeddings) and returns grounded answers with [S#] citations and char-span provenance.

- **Agents** — the review DAG runs analysis agents over a paper.
- **Report** — `report.py` turns agent results into a self-contained HTML+JSON report.

4.2 The async agent runtime

The review is a small, dependency-ordered team of agents:

- `agents/base.py` — the Agent ABC (`name`, `role`, `depends_on`, `async_run(ctx)`), plus `AgentContext` (holds `paper_id`, `bus`, a shared data blackboard) and `AgentResult`.
- `agents/orchestrator.py` — `run_agents()` validates the DAG (Kahn's algorithm; cycle/duplicate/unknown-dependency checks) and runs every ready agent concurrently via `asyncio.gather`. **Failure isolation is the contract:** one agent's exception never kills the run; dependents of a failed agent are skipped with an explicit "dependency failed: <dep>" reason.
- `agents/bus.py` — an in-process async pub/sub Bus with full history, mirroring every event to an optional append-only `EventLog`.
- `agents/events.py` — typed events (`AgentStarted`, `Finding`, `AgentDone`, `AgentError`, plus Lab pipeline events) and the JSONL `EventLog` audit trail.

`cli.py::_cmd_review` builds the agent list, wires a `Bus(log=EventLog(...))`, and runs `asyncio.run(run_agents(...))`, persisting the report and a per-run JSONL log under `~/.research-companion/.../runs/`.

4.3 Persistence — JSON store, no database

`store.py` keeps all state as files under `RESEARCH_COMPANION_DIR` (default `~/.research-companion/`). Global state (`.env`, `settings.json`, workspace registry) lives at the root; each workspace lives under `workspaces/<id>/`:

- `papers/<id-dirname>/` — one dir per paper: `paper.pdf`, `metadata.json`, `text.txt`, `extraction.json`, plus cached sections, structure, alignment, strength, embeddings, gaps, and the review report.
- `graph.json` (merged `NetworkX` graph) + `graph.html` (viz) at the workspace root.
- `runs/*.jsonl` — per-review agent event logs.

A **paper record** (`PaperMetadata`) holds `paper_id`, `title`, `authors`, `year`, `abstract`, `source_url`, `arxiv_categories`, `added_at`, `parse_source` (`pypdfium` / `docling` / `docling+ocr`), and `ocr_used`. Files are written atomically (`tmp + os.replace`). There is **no SQL/NoSQL database**; the only datastore-like dependency is `NetworkX` (in-memory, serialized to JSON).

4.4 Retrieval internals

- **BM25 (in-memory)** — `rank.py`: Okapi BM25 ($k_1=1.5$, $b=0.75$), lowercase alphanumeric tokenization (≥ 3 chars, minus stopwords). Built fresh per query.
- **Chunking with char provenance** — `chunking.py::chunk_section` splits sections into ~1200-char overlapping windows (150 overlap) on paragraph/sentence boundaries; each chunk carries **absolute** `char_start/char_end` so `paper_text[char_start:char_end] == chunk_text` (exact reader highlighting).
- **Hybrid ranking** — `retrieve.py::rank_units` fuses $0.5 \cdot \text{BM25} + 0.5 \cdot \text{cosine}$ via per-pool min-max normalization; embeddings are optional HuggingFace vectors (`embed.py`, gated on `HF_TOKEN`, cached per sub-chunk). ****Deterministic degradation:**** with no token/vectors it falls back to raw BM25 order bit-for-bit and tags results mode: "bm25" vs "hybrid"; it never raises for embedding reasons.

4.5 The Lab server

- **App factory** — `lab_api.py::create_lab_app(bus, *, llm=None)` builds a FastAPI app; FastAPI/uvicorn are lazily imported so the base package needs neither.
- **Serving** — `serve_lab(port=8765)` runs uvicorn on `127.0.0.1`; `GET /` returns the SPA `index.html`; static assets mount at `/static`.
- **API surface** — ~50 `/api` endpoints: papers CRUD/upload/retry, draft + citations (resolve/link) + placement, sections, graph, ingest (+scan), jobs, failures, align, ask, compare, settings, suggestions, workspaces, saved views, gaps, temporal, journey, search, converse, and `GET /api/events` (SSE stream).
- **Frontend** — a hand-rolled vanilla-JS SPA under `lab/static/js/` (`store.js/reducer.js/router.js/main.js, sse.js, components/*, views/*`).

5. Feature catalog

5.1 CLI reference

<code>S2/local-PDF</code> paper(s); batch from file		
action (cached) + build graph		
ost for the next build		
e interactive HTML graph		
RAG Q&A with citations (one-shot or REPL)		
n over graph nodes		
ment + graph statistics		

	json\	obsidian]`
ar topic search / citation expansion		
nces vs CrossRef/OpenAlex		
orted p-values (Statcheck) + GRIM mean		
	Export the library as BibTeX/RIS	
/Mendeley .bib into the library		
X draft's \cite keys against a .bib		
	Run the MCP trust-layer server (verification tools for external agents)	
team		
-by-point reviewer replies		
/show the draft paper		
against the draft		
section Q&A with [S#] sources		
erlap + optional LLM narrative		
analysis / temporal overview		
	use`	Manage isolated research workspaces
	failures`	The web Lab (server, folder ingest, failure list)

5.2 Agents

Agent	name	LLM?	Produces
IngestAgent	ingest	deterministic	graph node/edge counts (seeds the blackboard)
CitationAgent	citation	deterministic (API)	verified / suspect / unverified references
PriorArtAgent	priorart	deterministic (API)	related papers mapped onto the graph
NoveltyAgent	novelty	LLM	per-claim novelty verdicts + verified evidence
ConfidenceAgent	confidence	deterministic	per-claim confidence score + uncertainty band
BenchmarkAgent	benchmark	deterministic	suggested evaluation benchmarks
StatSoundnessAgent	statsoundness	deterministic	recomputed p-values + GRIM mean checks
ReproducibilityAgent	reproducibility	deterministic	reproducibility level (high/med/low) + gaps
EthicsAgent	ethics	deterministic	present / missing integrity declarations
SeverityAgent	severity	deterministic	findings ranked critical/major/minor

Agent	name	LLM?	Produces
VenueFitAgent	venuefit	LLM (grounded)	venue-fit verdict + desk-reject risk
RebuttalAgent	rebuttal	LLM	grounded reviewer replies + verified spans
ProblemStatementAgent	problem	LLM	refined problem statement (library-level)
TrackerAgent	tracker	deterministic	new-related-work sweep (library-level)

Dependencies are declared via `depends_on`; review wires ingest + citation + priorart + statsoundness + reproducibility + ethics always-on, adds novelty + confidence + benchmark + severity unless `--fast`, and adds venuefit when `--venue` is given.

5.3 The Research Lab UI

- **Views** — Home (journey + next-best-action), Library (cards/list, filters), Graph (live vis.js growth, Draft/Explore modes), Draft (alignment cards), Ask, Compare, Timeline (+ gaps), Suggestions, Settings, Researches.
- **Panels/components** — Citation Coverage (`citationsPanel.js`), Citation Placement (`placementPanel.js`), Reader (`reader.js`, char-offset highlight), Companion chat (`conversePanel.js`), Suggestions (`suggestionsPanel.js`), ingest modal + progress dock, welcome/key-prompt dialog, workspace switcher, toasts.

5.4 Key end-user features

- **Ingestion pipeline** — pluggable parser layer; `pypdfium2` default, optional Docling for layout/reading-order/tables/figures and forced full-page OCR. Docling runs in an **isolated subprocess** so a native crash/timeout falls back to `pypdfium` instead of killing the Lab; truncated parses fall back too. Folder ingest with pre-scan + per-file selection.
- **Library / dedup / metadata** — cross-source de-duplication (same paper via arXiv and as a local PDF); auto-extracted title/authors/year with a "Needs metadata" fallback and manual edit; OCR badge.
- **Citation coverage & placement** — your draft's bibliography as ground truth (in-library vs missing, one-click add, partial-coverage disclaimer); placement checks each `[n]` and author-year citation is in the right section.
- **Quote-grounded Q&A** — section-scoped answers with `[S#]` citations; verbatim quote verification surfaces `unverified_quotes`; click a citation to open the reader at the exact char span.
- **Alignment & strength** — per-section strengthens/challenges/alternative verdicts with verified evidence quotes; deterministic strength scoring.

- **Reviewer-grade checks (0.6)** — venue-fit, severity ranking, reproducibility, and integrity declarations (see §5.2 and `RELEASE_0.6.md`).
- **Rebuttal** — splits reviews, groups duplicate concerns, quotes only real passages (unground spans flagged), assembles a planned-revisions changelog.
- **Reports** — self-contained `report.html` + `report.json`; live SSE dashboard.
- **Interoperability** — BibTeX/RIS export of the library (`export-bib`), `.bib` import from Zotero/Mendeley (`import-bib`), and LaTeX `\cite`-key resolution against a `.bib` (`cite-tex`) — deterministic, in `research_companion/interop/`.
- **MCP trust-layer** — `research-companion mcp serve` exposes four deterministic, key-free tools (`verify_citation`, `ground_claim`, `citation_coverage`, `search_library`) to external agents over MCP. Logic in `research_companion/mcp_tools.py`; SDK wiring in `mcp_server.py` (optional `[mcp]` extra, lazily imported); versioned schemas in `docs/mcp-schemas/`.

6. Tech stack & dependencies

- **Build:** `setuptools ≥68`. **Python:** `>=3.10, <3.14`.
- **Runtime deps:** `anthropic>=0.40`, `openai>=1.40`, `pypdfium2>=4`, `httpx>=0.27`, `networkx>=3.0`, `jinja2>=3.1`, `feedparser>=6.0`.
- **Optional extras:** `server/demo = fastapi>=0.110 + uvicorn>=0.29`; `docling = docling>=2`; `mcp = mcp>=1.0` (the MCP trust-layer server); `dev = pytest, pytest-asyncio, ruff, fastapi, uvicorn`.
- **Entry point:** `research-companion = research_companion.cli:main`.
- **Version:** single source of truth in `pyproject.toml`; `__init__.py` reads installed metadata so `--version` never drifts.
- **Tests:** `pytest (testpaths=["tests"], asyncio_mode="auto", a slow marker)` — 1,780+ Python tests; `node --test tests/js/*.test.mjs` — 638 JS tests.
- **Lint:** `ruff (line-length 110, E501 ignored, select E, F, I, B, UP, SIM)`.

7. Design principles

- **Local-first** — plain files, in-memory graph, no DB, no required server.
- **Deterministic-first** — deterministic engines are foregrounded; retrieval degrades bit-for-bit to pure BM25 when embeddings are unavailable.
- **Verifiability** — grounded answers, verbatim quote checks, `refcheck` against authoritative records, and a typed append-only event audit trail.
- **Char-span provenance** — absolute offsets thread from chunking → Q&A → reader so the exact evidence span is highlightable.
- **Honesty boundaries** — failures surface honestly (`text_quality`,

IngestFailed, FAILED lanes); checks report only what they compute and skip ambiguous cases rather than guess.

8. Roadmap — where this is going

Shipped through **0.7.0**: the full novelty MVP (Phase 1), reviewer critique + venue fit (Phase 2), most of universal reach + integrity (Phase 3 — venue KB, reproducibility, integrity declarations), a deterministic **statistical soundness** checker (Statcheck + GRIM), **interoperability** (research_companion/interop/: BibTeX/RIS export, .bib import, LaTeX \cite-key resolution), and the ****MCP trust-layer server**** (research_companion/mcp_server.py, optional [mcp] extra, research-companion mcp serve) exposing four deterministic, key-free tools — `verify_citation`, `ground_claim`, `citation_coverage`, `search_library` — so any MCP-capable agent can *verify* against the library. See ROADMAP.md.

Planned next releases:

- **0.7.1 — Cost-gated MCP tools** — `ask_library / review_draft` behind an explicit budget/keys boundary; domain connectors (PubMed/Europe PMC/DBLP).
- **0.7.x — Plagiarism / near-duplicate detection** — completing the one open Phase-3 item, behind its own design gate (external similarity corpus + privacy model). The *integrity-declaration* side already ships.

Out of scope (non-goals): misconduct/fraud claims, semantic entailment, autonomous code execution, cloud/multi-user, GRIMMER/SPRITE (SD-level) and full LaTeX rendering.